

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Северо-Кавказский федеральный университет»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

КУРСОВАЯ РАБОТА

по дисциплине

«ИНТЕРНЕТ ТЕХНОЛОГИИ»

на тему:

«Разработка веб-приложения для планирования поездок»

Выполнил:

Кравченко Владислав Витальевич

Студент 4 курса группы ПИН-б-з-22-1

Направления подготовки

09.03.03 Прикладная информатика

заочной формы обучения

(подпись)

Руководитель работы:

(ФИО, должность)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работы выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

Ставрополь, 2026 г.

УТВЕРЖДАЮ

Директор департамента цифровых,
робототехнических систем
и электроники

Азаров

И.В.

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Направление подготовки 09.03.03 Прикладная информатика

Направленность (профиль) Прикладная информатика в экономике

ЗАДАНИЕ на курсовую работу

студента Кравченко Владислава Витальевича

по дисциплине Интернет-технологии

1. Тема работы Разработка веб-приложения для планирования поездок

2. Цель разработать полноценное веб-приложение для планирования туристических поездок, включающее клиентскую и серверную части, систему аутентификации, инструменты для составления маршрута, учёта расходов и формирования списка вещей.

3. Технологический стек: Python, Django, Django REST Framework, React, TypeScript, Docker.

4. Перечень подлежащих разработке вопросов:

а) по теоретической части: анализ предметной области планирования поездок, обзор существующих решений, обоснование выбора технологического стека, принципы построения SPA-приложений и REST API.

б) по практической части: проектирование моделей данных (Trip, ItineraryItem, Expense, PackingItem, Booking, UserSettings), реализация backend на Django REST Framework с JWT-аутентификацией, разработка frontend на React с TypeScript, реализация детальной страницы поездки с вкладками маршрута, расходов, чемодана, бронирований, карты и рекомендаций, поддержка тёмной и светлой темы, локализация

(русский/английский), выбор валюты и единиц измерения, контейнеризация с Docker и развёртывание на VPS.

5. Исходные данные: техническое задание на разработку веб-приложения для планирования поездок, документация по Django, Django REST Framework, React, TypeScript, Tailwind CSS, Docker.

6. Список рекомендуемой литературы: официальная документация Django, Django REST Framework, React, TypeScript, Tailwind CSS, Docker.

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % -	“ ”	20	г.
50 % -	“ ”	20	г.
75 % -	“ ”	20	г.
100 % -	“ ”	20	г.

8. Срок защиты студентом курсовой работы “ ” 20_ г.

Дата выдачи задания “ ” 20_ г.

Руководитель курсовой работы

(ученая степень, звание) (личная подпись) (инициалы, фамилия)

Задание принял(а) к исполнению студент _____ формы обучения
_____ курса _____ группы _____

(личная подпись) / _____
(инициалы, фамилия)

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Постановка задачи.....	8
1.2 Обзор существующих решений.....	9
1.3 Обоснование выбора технологии разработки	10
2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ.....	12
2.1 Функциональные требования.....	12
2.2 Нефункциональные требования	14
3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	15
3.1 Архитектура системы	15
3.2 Проектирование моделей данных	17
3.3 Проектирование REST API	21
3.4 Проектирование пользовательского интерфейса.....	23
4 РЕАЛИЗАЦИЯ СЕРВЕРНОЙ ЧАСТИ	24
4.1 Технологический стек backend	24
4.2 Реализация моделей и сериализаторов	26
4.3 Реализация ViewSet и изоляция данных	27
4.4 JWT-аутентификация и настройки пользователя	29
5 РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ.....	32
5.1 Технологический стек frontend	32
5.2 Реализация детальной страницы поездки.....	34
5.3 Темы оформления и локализация.....	36
5.4 Контейнеризация и развёртывание	39

6 ТЕСТИРОВАНИЕ	40
6.1 Методика тестирования.....	40
6.2 Результаты функционального тестирования.....	40
ЗАКЛЮЧЕНИЕ	44
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	46

ВВЕДЕНИЕ

В современном мире путешествия стали неотъемлемой частью жизни миллионов людей. Развитие транспортной инфраструктуры, доступность авиаперелётов и распространение онлайн-сервисов бронирования сделали туризм массовым явлением. Однако эффективная организация поездки требует учёта множества факторов: составления маршрута, планирования бюджета, учёта расходов, формирования списка необходимых вещей, бронирования билетов и отелей. Ручное планирование, основанное на записях в блокнотах или электронных таблицах, часто приводит к ошибкам, забыванию важных деталей и недопониманию между участниками поездки.

Веб-приложения для планирования поездок позволяют систематизировать весь процесс подготовки к путешествию, обеспечивая централизованное хранение информации, удобный доступ с любого устройства и возможность совместной работы. Современные решения предоставляют инструменты для составления маршрутов, учёта расходов, управления бронированиями и формирования списков вещей, что существенно повышает качество подготовки к поездке и снижает вероятность ошибок.

Объектом исследования является процесс разработки клиент-серверных веб-приложений с использованием современных фреймворков и технологий контейнеризации.

Предметом исследования выступают методы и средства создания веб-приложения для планирования поездок, включая проектирование REST API, реализацию одностраничного интерфейса (SPA), организацию аутентификации на основе JWT, поддержку тем оформления и локализации, а также контейнеризацию приложения с использованием Docker.

Целью курсовой работы является разработка полноценного веб-приложения для планирования туристических поездок, включающего клиентскую и серверную части, систему аутентификации, инструменты для составления маршрута, учёта расходов и формирования списка вещей, а также поддержку бронирований, карты и рекомендаций.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и обзор существующих решений для планирования поездок;
- сформулировать функциональные и нефункциональные требования к разрабатываемому приложению;
- спроектировать архитектуру системы на основе разделения ответственности между backend и frontend;
- спроектировать модели данных (Trip, ItineraryItem, Expense, PackingItem, Booking, UserSettings) и REST API;
- реализовать backend-часть на языке Python с использованием Django и Django REST Framework;
- реализовать frontend-часть на языке TypeScript с использованием React и Vite;
- реализовать детальную страницу поездки с шестью вкладками: маршрут, расходы, чемодан, бронирования, карта и рекомендации;
- организовать аутентификацию и авторизацию пользователей на основе JWT-токенов;
- реализовать поддержку тёмной и светлой темы оформления;

- реализовать локализацию интерфейса (русский и английский языки);
- реализовать выбор валюты, единиц расстояния и температуры;
- выполнить контейнеризацию приложения с использованием Docker и Docker Compose;
- развернуть приложение на VPS-сервере с использованием Nginx и SSL-сертификатов;
- провести функциональное тестирование основных сценариев работы приложения.

Практическая значимость работы заключается в том, что разработанное приложение может быть использовано в качестве готового решения для планирования личных и командных поездок, а также в качестве учебного примера построения современных клиент-серверных веб-приложений с применением актуального технологического стека.

Структура работы. Курсовая работа состоит из введения, шести глав, заключения и списка использованных источников. В первой главе проводится анализ предметной области. Во второй главе формулируется техническое задание. В третьей главе рассматриваются вопросы проектирования системы. Четвёртая глава посвящена реализации серверной части. Пятая глава — реализации клиентской части и развёртыванию. В шестой главе описываются результаты тестирования. Объём работы составляет 30–40 страниц.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Постановка задачи

Планирование поездки — это комплексный процесс, требующий учёта множества аспектов. Путешественнику необходимо определить маршрут и хронологию посещения мест, рассчитать бюджет и отслеживать расходы в процессе поездки, составить список необходимых вещей и отметить уже собранные, забронировать билеты и отели, а также сохранить всю информацию в удобном и доступном виде. Традиционные подходы к планированию (записи в блокноте, заметки в телефоне, электронные таблицы) обладают рядом существенных недостатков: фрагментарностью информации, сложностью обновления, отсутствием структурирования и ограниченными возможностями доступа с разных устройств.

Современные веб-приложения для планирования поездок лишены указанных недостатков. Они обеспечивают централизованное хранение всех данных о поездке, удобный доступ с любого устройства через веб-браузер, структурированное представление информации (маршрут, расходы, чемодан, бронирования) и возможность настройки под индивидуальные предпочтения пользователя (язык интерфейса, тема оформления, валюта, единицы измерения).

В рамках данной курсовой работы разрабатывается веб-приложение для планирования поездок, которое должно сочетать простоту использования с современными подходами к разработке программного обеспечения. Приложение должно предоставлять пользователю возможность создавать поездки, указывать их название, направление, даты и бюджет, составлять маршрут с пунктами посещения, учитывать расходы по категориям, формировать список вещей с отметкой собранных, сохранять информацию о бронированиях (авиабилеты, отели, ж/д, аренда авто) и настраивать параметры интерфейса.

Разрабатываемое приложение должно быть построено по клиент-серверной архитектуре с разделением ответственности между backend, реализующим REST API, и frontend, реализующим пользовательский интерфейс в виде одностраничного приложения (SPA). Такое разделение обеспечивает независимость развития клиентской и серверной частей, а также упрощает масштабирование и поддержку системы.

Дополнительно приложение должно поддерживать настройку параметров интерфейса: выбор языка (русский или английский), темы оформления (светлая или тёмная), валюты (российский рубль, доллар США, евро, британский фунт, японская йена), единиц расстояния (километры или мили) и температуры (градусы Цельсия или Фаренгейта). Эти настройки должны сохраняться как на клиенте (в localStorage), так и на сервере (в базе данных), что обеспечивает их восстановление при входе с любого устройства.

1.2 Обзор существующих решений

На рынке программного обеспечения для планирования поездок представлен широкий спектр решений, различающихся по функциональности, стоимости и целевой аудитории. Среди наиболее известных можно выделить следующие.

Google Travel — сервис от Google, интегрированный с Gmail и Google Maps. Предоставляет возможность планирования маршрутов, бронирования отелей и авиабилетов, а также сохранения информации о поездках. Отличается глубокой интеграцией с экосистемой Google, однако является закрытым облачным сервисом без возможности самостоятельного развёртывания.

TripIt — популярный сервис планирования поездок, автоматически создающий маршрут на основе электронных писем с подтверждениями бронирований. Предоставляет единое расписание поездки, доступное

онлайн и офлайн. Является коммерческим продуктом с ограниченной бесплатной версией.

Wanderlog — приложение для планирования поездок с поддержкой совместной работы, карт, заметок и бюджета. Отличается современным интерфейсом и широким функционалом, однако является коммерческим облачным сервисом.

Roadtrippers — специализированный сервис для планирования автомобильных путешествий, предоставляющий карты, расчёт маршрутов и точки интереса вдоль пути. Ориентирован преимущественно на авторские поездки.

Анализ существующих решений позволяет сформулировать следующие выводы. Во-первых, большинство популярных сервисов являются облачными и не предоставляют возможности самостоятельного развёртывания. Во-вторых, многие из них имеют ограниченную бесплатную версию и требуют платной подписки для доступа к полному функционалу. В-третьих, существующие решения часто перегружены функциями, которые могут быть избыточны для простого планирования личных поездок. Указанные факторы обуславливают целесообразность разработки собственного веб-приложения, которое сочетало бы необходимый функционал с современным технологическим стеком и возможностью самостоятельного развёртывания.

1.3 Обоснование выбора технологии разработки

Для реализации веб-приложения был выбран современный технологический стек, обеспечивающий высокую производительность, удобство разработки и возможность дальнейшего масштабирования.

В качестве языка программирования для backend-части выбран Python 3.10 — один из наиболее популярных языков программирования, отличающийся простым и выразительным синтаксисом, богатой стандартной библиотекой и широким выбором сторонних пакетов. Для

реализации backend выбран фреймворк Django 5, предоставляющий встроенные средства для работы с базой данных (ORM), аутентификацией, маршрутизацией URL и администрированием. Django следует принципу «batteries included», что существенно ускоряет процесс разработки.

Для реализации REST API поверх Django используется Django REST Framework (DRF) — мощный и гибкий инструмент, предоставляющий сериализаторы, ViewSet, маршрутизаторы (DefaultRouter) и средства аутентификации. DRF является стандартом де-факто для построения REST API в экосистеме Django.

Для аутентификации используется JWT (JSON Web Token) с пакетом djangorestframework-simplejwt. JWT обеспечивает stateless-аутентификацию, при которой токен содержит всю необходимую информацию о пользователе и не требует хранения состояния сессии на сервере. Это хорошо сочетается с REST-архитектурой и упрощает масштабирование.

В качестве языка программирования для frontend-части выбран TypeScript — статически типизированное надмножество JavaScript. Использование TypeScript позволяет выявлять ошибки на этапе компиляции, улучшает читаемость кода и упрощает его поддержку. Для построения пользовательского интерфейса выбрана библиотека React 18, обеспечивающая компонентный подход и эффективное обновление DOM за счёт виртуального DOM.

Для сборки frontend-проекта используется Vite 5 — современный сборщик, обеспечивающий высокую скорость разработки за счёт использования нативных ES-модулей. Для стилизации применяется Tailwind CSS 3 — utility-first CSS-фреймворк, позволяющий создавать дизайн непосредственно в разметке компонентов.

Для контейнеризации приложения используются Docker и Docker Compose, обеспечивающие изоляцию среды выполнения, воспроизводимость сборки и удобство развёртывания. Для развёртывания в

production применяется Nginx в качестве обратного прокси и веб-сервера для раздачи статических файлов, а также Let's Encrypt для получения бесплатных SSL-сертификатов.

Для контроля версий используется Git с размещением репозитория на платформе GitHub, что обеспечивает совместную работу, отслеживание изменений и возможность отката к предыдущим версиям.

Выбранный стек технологий является современным, широко распространённым и хорошо документированным, что обеспечивает возможность дальнейшего развития проекта и его поддержки.

2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1 Функциональные требования

1. Регистрация и аутентификация пользователей. Приложение должно обеспечивать регистрацию новых пользователей по логину, email и паролю, а также вход по логину и паролю. Аутентификация должна реализовываться на основе JWT-токенов (access и refresh). При регистрации должны автоматически создаваться настройки пользователя со значениями по умолчанию.
2. Управление поездками. Приложение должно обеспечивать выполнение следующих операций с поездками:
 - создание поездки с указанием названия, описания, направления, дат начала и окончания, бюджета и статуса;
 - редактирование поездки;
 - удаление поездки (с каскадным удалением всех связанных данных);
 - просмотр списка поездок пользователя;
 - просмотр детальной информации о поездке.

3. Маршрут (itinerary). Приложение должно предоставлять возможность добавлять пункты маршрута с указанием даты, времени, названия, описания, места и порядка. Пользователь должен иметь возможность удалять пункты маршрута.

4. Расходы (expenses). Приложение должно обеспечивать:

- добавление расходов с указанием названия, суммы, категории и даты;
- удаление расходов;
- автоматический расчёт общей суммы расходов по поездке;
- отображение отношения потраченной суммы к бюджету;
- категоризацию расходов (транспорт, жильё, еда, развлечения, покупки, другое).

5. Чемодан (packing). Приложение должно предоставлять возможность:

- добавлять вещи в список с указанием названия;
- отмечать вещи как упакованные (чекбокс);
- удалять вещи из списка.

6. Бронирования (bookings). Приложение должно обеспечивать:

- добавление бронирований с указанием типа (авиабилет, отель, ж/д, аренда авто, другое), названия, номера подтверждения, места, цены и дат;
- удаление бронирований;
- функцию парсинга email с подтверждением бронирования (заглушка для демонстрации).

7. Настройки пользователя. Приложение должно предоставлять возможность настройки:

- языка интерфейса (русский, английский);
- темы оформления (светлая, тёмная);
- валюты (рубль, доллар, евро, фунт, йена);
- единиц расстояния (километры, мили);

- единиц температуры (Цельсий, Фаренгейт);
 - email- и push-уведомлений.
8. Изоляция данных. Пользователь должен иметь доступ только к своим поездкам и связанным с ними данным.

2.2 Нефункциональные требования

1. Архитектура. Приложение должно быть построено по клиент-серверной архитектуре с разделением ответственности между backend (Django REST API) и frontend (React SPA).
2. Технологии. Backend должен быть реализован на Python 3.10 с использованием Django 5 и Django REST Framework. Frontend должен быть реализован на TypeScript с использованием React 18 и Vite. Для стилизации должен использоваться Tailwind CSS.
3. Контейнеризация. Приложение должно быть упаковано в Docker-контейнеры. Для оркестрации должен использоваться Docker Compose. Должны быть предусмотрены отдельные конфигурации для разработки и для production.
4. Веб-сервер. В production должен использоваться Nginx в качестве обратного прокси и для раздачи статических файлов frontend. Backend должен запускаться под управлением gunicorn.
5. Безопасность. Доступ к API должен быть защищён JWT-аутентификацией. В production должна быть включена поддержка HTTPS. SECRET_KEY должен передаваться через переменные окружения.
6. Локализация. Пользовательский интерфейс должен поддерживать русский и английский языки. Выбранный язык должен сохраняться в настройках пользователя.
7. Темы оформления. Приложение должно поддерживать светлую и тёмную темы. Выбранная тема должна сохраняться в localStorage и синхронизироваться с настройками на сервере.

8. Адаптивность. Пользовательский интерфейс должен быть адаптивным и корректно отображаться на мобильных и десктоп-устройствах.
9. Воспроизводимость. Проект должен храниться в системе контроля версий Git. Сборка и развёртывание должны быть автоматизированы с использованием Docker.

3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

3.1 Архитектура системы

Разрабатываемое приложение построено по клиент-серверной архитектуре с чётким разделением ответственности между серверной (backend) и клиентской (frontend) частями. Такой подход является стандартом для современных веб-приложений и обеспечивает независимое развитие клиентской и серверной частей, возможность использования разных языков программирования и инструментов, а также упрощение тестирования и масштабирования.

Архитектура веб-приложения TravelPlanner

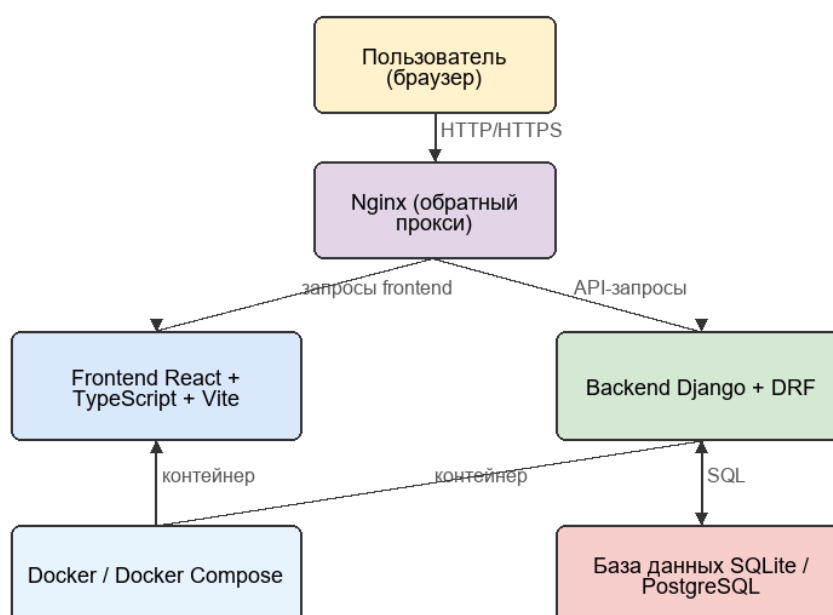


Рисунок 1 — Общая архитектура приложения

Серверная часть (backend) реализована на языке Python 3.10 с использованием фреймворка Django 5 и расширения Django REST Framework. Backend предоставляет REST API, обеспечивающий выполнение всех бизнес-операций: аутентификацию пользователей, управление поездками, маршрутами, расходами, чемоданом, бронированиями и настройками. Backend не формирует HTML-разметку интерфейса — его единственной обязанностью является обработка запросов и возврат данных в формате JSON.

Клиентская часть (frontend) реализована на языке TypeScript с использованием библиотеки React 18 и сборщика Vite. Frontend представляет собой одностраничное приложение (SPA), которое загружается один раз и далее взаимодействует с сервером исключительно через REST API. Такой подход обеспечивает высокую скорость работы приложения, так как при навигации между страницами не происходит полной перезагрузки документа.

Взаимодействие между frontend и backend осуществляется по протоколу HTTP с использованием формата JSON. Для аутентификации запросов используется схема Bearer Token с JWT-токенами. Все запросы к API, за исключением endpoints регистрации, входа и обновления токена, требуют предъявления действительного access-токена.

В качестве базы данных используется SQLite — лёгкая встроенная СУБД, не требующая отдельного сервера. В файле зависимостей присутствует драйвер psycopg2-binary, обеспечивающий возможность перехода на PostgreSQL в production-окружении.

Архитектура приложения в production-окружении включает следующие компоненты:

- Nginx — обратный прокси, терминирование SSL, раздача статических файлов frontend, проксирование запросов к /api/ на backend;
- gunicorn — WSGI-сервер, запускающий Django-приложение;
- Django + DRF — серверная логика и REST API;
- SQLite — хранение данных;
- React SPA — клиентская часть, собираемая Vite и раздаваемая Nginx.

Контейнеризация обеспечивает изоляцию компонентов и воспроизводимость окружения. Backend и frontend размещаются в отдельных Docker-контейнерах, а их взаимодействие организуется через внутреннюю сеть Docker. Для хранения загруженных файлов (обложки поездок) используется общий именованный том (volume), доступный обоим контейнерам.

3.2 Проектирование моделей данных

Для хранения данных приложения спроектированы шесть моделей, организованных в два Django-приложения: accounts (аутентификация и настройки) и trips (поездки и связанный контент).

Таблица 1 — Поля модели Trip

Поле	Тип данных	Описание
id	INTEGER PK	Уникальный идентификатор поездки
title	VARCHAR(200)	Название поездки
destination	VARCHAR(200)	Место назначения
start_date	DATE	Дата начала
end_date	DATE	Дата окончания
user_id	INTEGER FK	Создатель поездки
created_at	DATETIME	Дата создания

ER-диаграмма базы данных TravelPlanner

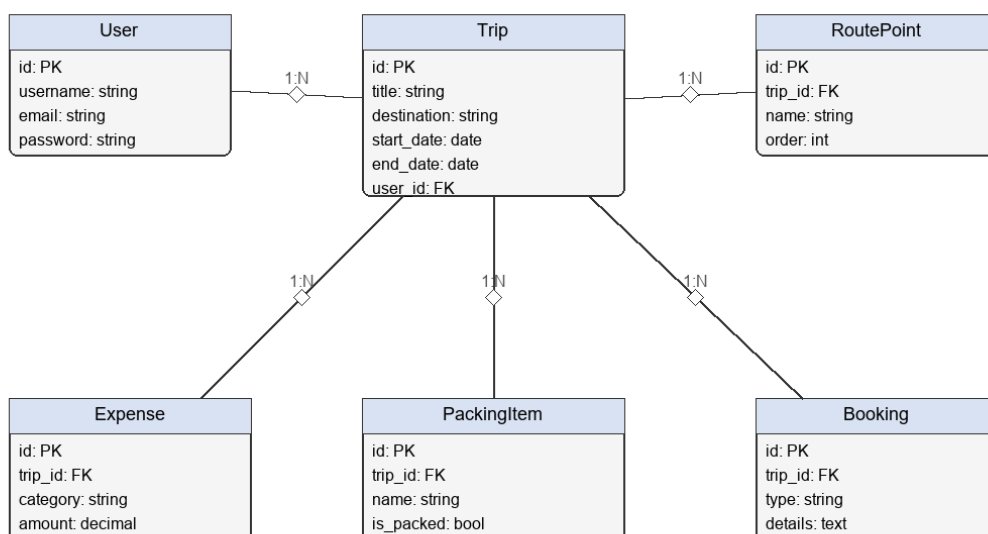


Рисунок 2 — Логическая модель данных

В качестве модели пользователя используется стандартная модель `django.contrib.auth.models.User`, что обеспечивает поддержку регистрации, аутентификации, хеширования паролей и разграничения доступа без необходимости реализации этих механизмов с нуля.

Модель `UserSettings` расширяет стандартную модель пользователя и предназначена для хранения настроек интерфейса. Модель связана с `User` отношением «один к одному» (`OneToOneField`). Настройки создаются автоматически при регистрации пользователя.

Код модели `UserSettings`:

```

class UserSettings(models.Model):
    LANGUAGE_CHOICES = [("ru", "Русский"), ("en", "English")]
    THEME_CHOICES =
[("light", "Светлая"), ("dark", "Тёмная")]
    CURRENCY_CHOICES = [("RUB", "₽ Рубль"), ("USD", "$
Доллар"),
                        ("EUR", "€ Евро"), ("GBP", "£ Фунт"),
                        ("JPY", "¥ Йена")]
    DISTANCE_CHOICES = [("km", "Километры"), ("mi", "Мили")]
    TEMPERATURE_CHOICES = [("C", "°C"), ("F", "°F")]

    user = models.OneToOneField(User,
on_delete=models.CASCADE,
                                related_name="settings")
    language = models.CharField(max_length=2,
choices=LANGUAGE_CHOICES,
                                default="ru")
    theme = models.CharField(max_length=10,
choices=THEME_CHOICES,
                                default="light")
    currency = models.CharField(max_length=3,
choices=CURRENCY_CHOICES,
                                default="RUB")
    distance_unit = models.CharField(max_length=2,
choices=DISTANCE_CHOICES,
                                default="km")
    temperature_unit = models.CharField(max_length=1,
choices=TEMPERATURE_CHOICES,
                                default="C")
    email_notifications =
models.BooleanField(default=True)
    push_notifications = models.BooleanField(default=True)
    google_maps_api_key = models.CharField(max_length=255,
blank=True)

```

Модель Trip является центральной сущностью приложения и содержит: title, description, start_date, end_date, destination, budget (DecimalField), status (planned, active, completed, cancelled), cover_image (ImageField), created_at, updated_at. Связана с пользователем отношением «один ко многим» (ForeignKey, CASCADE).

Код модели Trip:

```

class Trip(models.Model):
    STATUS_CHOICES = [("planned", "Запланирована"),
                    ("active", "Активна"),
                    ("completed", "Завершена"),
                    ("cancelled", "Отменена")]
    user = models.ForeignKey(User,

```

```

on_delete=models.CASCADE,
                                related_name="trips")
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    start_date = models.DateField()
    end_date = models.DateField()
    destination = models.CharField(max_length=200)
    budget = models.DecimalField(max_digits=10,
                                decimal_places=2,
                                default=0)
    status = models.CharField(max_length=20,
choices=STATUS_CHOICES,
                                default="planned")
    cover_image = models.ImageField(upload_to="trips/",
                                blank=True, null=True)

    class Meta:
        ordering = ["-created_at"]

```

Модель `ItineraryItem` (пункт маршрута) содержит: `date`, `time`, `title`, `description`, `location`, `order`. Связана с `Trip` отношением «один ко многим» (CASCADE). Сортировка — по дате, времени и порядку.

Модель `Expense` (расход) содержит: `title`, `amount` (`DecimalField`), `category` (`transport`, `housing`, `food`, `entertainment`, `shopping`, `other`), `date`. Связана с `Trip` (CASCADE).

Модель `PackingItem` (вещь в чемодане) содержит: `name`, `is_packed` (`BooleanField`, `default False`). Связана с `Trip` (CASCADE).

Модель `Booking` (бронирование) содержит: `booking_type` (`flight`, `hotel`, `train`, `car`, `other`), `title`, `confirmation_number`, `start_date`, `end_date`, `location`, `price`, `email_source`, `is_parsed`. Связана с `Trip` (CASCADE). Поля `email_source` и `is_parsed` подготовлены для будущей интеграции парсинга email-подтверждений.

Структура связей между моделями:

- User 1:1 UserSettings (настройки пользователя);
- User 1:N Trip (поездки пользователя);
- Trip 1:N ItineraryItem (пункты маршрута);
- Trip 1:N Expense (расходы);
- Trip 1:N PackingItem (вещи в чемодане);
- Trip 1:N Booking (бронирования).

Все дочерние сущности используют `on_delete=CASCADE`, что обеспечивает удаление связанных данных при удалении поездки. Доступ к данным строго ограничен владельцем: каждый запрос фильтруется по полю `user=request.user`.

3.3 Проектирование REST API

REST API приложения спроектировано в соответствии с принципами REST и состоит из двух групп endpoints: аутентификация (приложение `accounts`) и управление данными поездок (приложение `trips`).

Таблица 2 — Основные endpoint'ы REST API

Метод	Endpoint	Назначение
POST	<code>/api/token/</code>	Получение JWT-токенов
POST	<code>/api/token/refresh/</code>	Обновление access-токена
GET	<code>/api/trips/</code>	Список поездок
POST	<code>/api/trips/</code>	Создание поездки
GET	<code>/api/trips/{id}/</code>	Детали поездки
PATCH	<code>/api/trips/{id}/</code>	Обновление поездки
DELETE	<code>/api/trips/{id}/</code>	Удаление поездки
GET	<code>/api/trips/{id}/route/</code>	Маршрут поездки
GET	<code>/api/trips/{id}/expenses/</code>	Расходы поездки

Endpoints аутентификации (префикс `/api/auth/`):

1. POST `/api/auth/register/` — регистрация нового пользователя.

Создаёт пользователя и автоматически создаёт `UserSettings` со значениями по умолчанию.

2. POST `/api/auth/login/` — вход пользователя. Возвращает пару JWT-токенов (`access` и `refresh`).

3. POST `/api/auth/refresh/` — обновление access-токена по refresh-токену.

4. GET `/api/auth/profile/` — получение данных текущего пользователя с вложенными настройками.

5. GET, PATCH `/api/auth/settings/` — получение и обновление настроек пользователя.

Endpoints управления поездками (префикс `/api/`, используется `DefaultRouter`):

6. `/api/trips/` — полный CRUD для поездок (GET список, POST создание, GET/PUT/PATCH/DELETE отдельная поездка).
7. `/api/itinerary/` — полный CRUD для пунктов маршрута.
8. `/api/expenses/` — полный CRUD для расходов.
9. `/api/packing/` — полный CRUD для вещей чемодана.
10. `/api/bookings/` — полный CRUD для бронирований.
11. `POST /api/parse-booking-email/` — парсинг email с подтверждением бронирования (демонстрационная заглушка).

Для реализации CRUD-операций используются `ViewSet` из `Django REST Framework`, наследуемые от `ModelViewSet`. `ModelViewSet` автоматически предоставляет реализацию операций `list`, `create`, `retrieve`, `update`, `partial_update` и `destroy`.

Важной особенностью реализации является переопределение метода `get_queryset` в каждом `ViewSet`, обеспечивающего изоляцию данных пользователей:

```
class TripViewSet(viewsets.ModelViewSet):
    permission_classes = (IsAuthenticated,)
    def get_queryset(self):
        return Trip.objects.filter(user=self.request.user)
    def get_serializer_class(self):
        if self.action == "list":
            return TripListSerializer
        return TripSerializer
    def perform_create(self, serializer):
        serializer.save(user=self.request.user)
```

Для дочерних сущностей фильтрация осуществляется через владельца поездки:

```
def get_queryset(self):
    return ItineraryItem.objects.filter(
        trip__user=self.request.user)
```

Это гарантирует, что пользователь не может получить доступ к данным чужих поездок. Для сериализации используются вложенные сериализаторы: `TripSerializer` включает вложенные `ItineraryItemSerializer`,

ExpenseSerializer, PackingItemSerializer и BookingSerializer, а также вычисляемое поле total_expenses (сумма всех расходов поездки).

Для списка поездок используется облегчённый TripListSerializer (без вложенных данных) для повышения производительности.

3.4 Проектирование пользовательского интерфейса

Пользовательский интерфейс приложения спроектирован в виде одностраничного приложения (SPA) на основе React. Интерфейс включает следующие страницы:

- страница входа (Login) — форма аутентификации с полями логина и пароля;
- страница регистрации (Register) — форма регистрации с полями логина, email и пароля;
- список поездок (Trips) — главная страница с карточками поездок;
- форма поездки (TripForm) — создание и редактирование поездки;
- детальная страница поездки (TripDetail) — основная страница с шестью вкладками;
- страница настроек (Settings) — управление параметрами интерфейса.

Навигация между экранами реализована с использованием react-router-dom версии 6. Защита приватных маршрутов осуществляется компонентом PrivateRoute, который перенаправляет неавторизованных пользователей на страницу входа.

Центральным элементом интерфейса является детальная страница поездки (TripDetail), содержащая шесть вкладок:

1. Маршрут (itinerary) — форма добавления пунктов маршрута (дата, время, название, место) и список пунктов.
2. Расходы (expenses) — форма добавления расходов (название, сумма, категория, дата) и список расходов с общей суммой.

3. Чемодан (packing) — поле ввода вещи и список с чекбоксами для отметки собранных вещей.
4. Бронирования (bookings) — форма добавления бронирований (тип, название, номер подтверждения, место, цена) и список бронирований с иконками по типу.
5. Карта (map) — плейсхолдер с приглашением добавить Google Maps API-ключ в настройках.
6. Рекомендации (recommendations) — поле ввода предпочтений и генерация рекомендаций на основе направления поездки.

Шапка страницы отображает название поездки, направление, даты, статус-бейдж, кнопки редактирования и удаления, а также блок бюджета с суммой расходов и запланированным бюджетом.

Компонент Layout обеспечивает общую раскладку: навигационную панель с логотипом, именем пользователя, переключателем темы и кнопками выхода. Для вложенных маршрутов используется Outlet из react-router-dom.

4 РЕАЛИЗАЦИЯ СЕРВЕРНОЙ ЧАСТИ

4.1 Технологический стек backend

Серверная часть приложения реализована на языке Python 3.10 с использованием фреймворка Django 5. Выбор Python обусловлен его выразительным синтаксисом, богатой экосистемой библиотек и широким распространением в области веб-разработки. Django предоставляет встроенные средства для работы с базой данных (ORM), аутентификацией, маршрутизацией URL, администрированием и обработкой форм, что существенно сокращает объём кода.

Для построения REST API используется Django REST Framework (DRF) — расширение, предоставляющее сериализаторы, ViewSet,

маршрутизаторы и средства аутентификации. DRF является стандартом де-факто для разработки REST API в экосистеме Django. Преимущества DRF:

- встроенная поддержка сериализации моделей в JSON;
- `ViewSet`, сокращающие объём кода для CRUD-операций;
- автоматическая генерация URL с помощью `DefaultRouter`;
- встроенная поддержка JWT-аутентификации;
- возможность детальной настройки разрешений (permissions).

Для JWT-аутентификации используется пакет `django-rest-framework-simplejwt`. Access-токен живёт 1 день, refresh-токен — 7 дней, что обеспечивает баланс между безопасностью и удобством пользователя.

Для обработки CORS используется пакет `django-cors-headers`, необходимый при раздельном развёртывании frontend и backend.

Для обработки изображений (обложек поездок) используется библиотека `Pillow`, необходимая для работы `ImageField`.

Для запуска в production используется `gunicorn` — WSGI-сервер, обеспечивающий высокую производительность.

Содержимое файла `requirements.txt`:

```
django>=5.0
djangorestframework
django-cors-headers
djangorestframework-simplejwt
psycopg2-binary
gunicorn
Pillow
```

Структура backend-приложения:

```
backend/
├── config/                # настройки проекта Django
│   ├── settings.py       # конфигурация
│   ├── urls.py           # корневой роутинг
│   └── wsgi.py           # WSGI-точка входа
├── accounts/             # аутентификация и настройки
│   ├── models.py         # UserSettings
│   ├── serializers.py
│   ├── views.py          # RegisterView, ProfileView
│   └── urls.py
├── trips/                # управление поездками
│   ├── models.py         # Trip, ItineraryItem, Expense,
│   │                     # PackingItem, Booking
│   └── serializers.py
```

```

├── views.py      # ViewSet'ы
├── admin.py
├── urls.py       # DefaultRouter
├── requirements.txt
├── Dockerfile
└── manage.py

```

Настройки DRF и JWT в файле settings.py:

```

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (

"rest_framework_simplejwt.authentication.JWTAuthentication
",
    ),
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
}
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(days=1),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
}

```

4.2 Реализация моделей и сериализаторов

Модели Django реализованы в двух приложениях: accounts (модель UserSettings) и trips (модели Trip, ItineraryItem, Expense, PackingItem, Booking). Подробное описание моделей приведено в разделе 3.2. Здесь рассмотрим реализацию сериализаторов.

Сериализаторы DRF обеспечивают преобразование объектов моделей в JSON и обратно. Для модели Trip используются два сериализатора: TripSerializer (с вложенными дочерними элементами) и TripListSerializer (облегчённый, для списка).

Код TripSerializer с вложенными данными:

```

class TripSerializer(serializers.ModelSerializer):
    itinerary_items = ItineraryItemSerializer(many=True,
read_only=True)
    expenses = ExpenseSerializer(many=True,
read_only=True)
    packing_items = PackingItemSerializer(many=True,
read_only=True)
    bookings = BookingSerializer(many=True,
read_only=True)
    total_expenses = serializers.SerializerMethodField()

```

```

class Meta:
    model = Trip
    fields = "__all__"
    read_only_fields = ("user",)

def get_total_expenses(self, obj):
    return sum(e.amount for e in obj.expenses.all())

```

Вычисляемое поле `total_expenses` использует `SerializerMethodField` и рассчитывает сумму всех расходов поездки. Это позволяет клиенту отображать отношение потраченной суммы к бюджету без дополнительных запросов.

`TripListSerializer` не включает вложенные данные, что существенно повышает производительность при отображении списка поездок:

```

class TripListSerializer(serializers.ModelSerializer):
    total_expenses = serializers.SerializerMethodField()
    class Meta:
        model = Trip
        fields = ("id", "title", "destination", "start_date",
                  "end_date", "status", "budget",
                  "total_expenses", "cover_image")

```

Для регистрации пользователя используется `RegisterSerializer`, который создаёт пользователя и автоматически создаёт `UserSettings` со значениями по умолчанию:

```

class RegisterSerializer(serializers.ModelSerializer):
    def create(self, validated_data):
        user = User.objects.create_user(
            username=validated_data["username"],
            email=validated_data.get("email", ""),
            password=validated_data["password"])
        UserSettings.objects.create(user=user)
        return user

```

4.3 Реализация `ViewSet` и изоляция данных

Для реализации CRUD-операций с поездками и связанными сущностями используются `ViewSet` из Django REST Framework. Каждый `ViewSet` наследуется от `ModelViewSet` и переопределяет метод `get_queryset` для обеспечения изоляции данных.

Реализация TripViewSet:

```
class TripViewSet(viewsets.ModelViewSet):
    permission_classes = (IsAuthenticated,)
    def get_queryset(self):
        return Trip.objects.filter(user=self.request.user)
    def get_serializer_class(self):
        if self.action == "list":
            return TripListSerializer
        return TripSerializer
    def perform_create(self, serializer):
        serializer.save(user=self.request.user)
```

Метод `get_serializer_class` выбирает сериализатор в зависимости от действия: для списка (`action == "list"`) используется облегчённый `TripListSerializer`, для остальных действий — полный `TripSerializer` с вложенными данными. Это оптимизирует производительность: при запросе списка поездок не загружаются все дочерние элементы.

Метод `perform_create` автоматически устанавливает владельца поездки из текущего пользователя, что исключает возможность подмены владельца.

Для дочерних сущностей (`ItineraryItem`, `Expense`, `PackingItem`, `Booking`) фильтрация осуществляется через владельца поездки:

```
class ItineraryItemViewSet(viewsets.ModelViewSet):
    permission_classes = (IsAuthenticated,)
    def get_queryset(self):
        return ItineraryItem.objects.filter(
            trip__user=self.request.user)
    def perform_create(self, serializer):
        trip = get_object_or_404(Trip,
            id=self.request.data.get("trip"),
            user=self.request.user)
        serializer.save(trip=trip)
```

Метод `perform_create` для дочерних сущностей проверяет, что указанная поездка принадлежит текущему пользователю, с помощью `get_object_or_404` с фильтром `user=request.user`. Если поездка не найдена (не принадлежит пользователю), возвращается ошибка 404, что предотвращает создание записей для чужих поездок.

Для маршрутизации используется `DefaultRouter`, который автоматически генерирует URL для всех CRUD-операций:

```
router = DefaultRouter()
router.register(r"trips", TripViewSet, basename="trip")
router.register(r"itinerary", ItineraryItemViewSet,
                basename="itinerary")
router.register(r"expenses", ExpenseViewSet,
                basename="expense")
router.register(r"packing", PackingItemViewSet,
                basename="packing")
router.register(r"bookings", BookingViewSet,
                basename="booking")
```

Дополнительно реализована функция `parse_booking_email` — демонстрационная заглушка для парсинга email-подтверждений бронирований. Функция принимает текст письма и ID поездки, создаёт запись бронирования с типом `flight` и помечает её как спарсенную. Полная интеграция требует подключения почтового сервиса (IMAP/API).

4.4 JWT-аутентификация и настройки пользователя

Аутентификация в приложении реализована на основе JWT (JSON Web Token) — стандарта создания токенов доступа, содержащих утверждения о пользователе и подписанных криптографически. Используется пакет `django-rest-framework-simplejwt`.

 **TripPlanner**

Вход в систему

Логин

Пароль

Войти

Нет аккаунта? [Зарегистрироваться](#)

Рисунок 3 — Форма авторизации пользователя

На рисунке 3 представлена форма авторизации пользователя. Здесь пользователь вводит логин и пароль для получения доступа к личному кабинету и функциям планирования поездок.

Процесс аутентификации:

1. Регистрация: пользователь отправляет POST-запрос на `/api/auth/register/` с логином, email и паролем. Создаётся пользователь с хешированным паролем и автоматически создаётся `UserSettings`.
2. Вход: пользователь отправляет POST-запрос на `/api/auth/login/` с логином и паролем. Сервер проверяет учётные данные и возвращает пару JWT-токенов (access и refresh).

3. Авторизованные запросы: frontend добавляет access-токен в заголовок `Authorization: Bearer <token>` ко всем последующим запросам.
4. Обновление токена: при истечении срока действия access-токена frontend отправляет refresh-токен на `/api/auth/refresh/` и получает новый access-токен.
5. Получение профиля: `GET /api/auth/profile/` возвращает данные пользователя с вложенными настройками.
6. Обновление настроек: `PATCH /api/auth/settings/` обновляет настройки пользователя (язык, тема, валюта, единицы измерения).

Реализация `UserSettingsView` использует `get_or_create` для ленивого создания настроек:

```
class UserSettingsView(APIView):
    permission_classes = (IsAuthenticated,)
    def get(self, request):
        settings, _ = UserSettings.objects.get_or_create(
            user=request.user)
        serializer = UserSettingsSerializer(settings)
        return Response(serializer.data)
    def patch(self, request):
        settings, _ = UserSettings.objects.get_or_create(
            user=request.user)
        serializer = UserSettingsSerializer(settings,
            data=request.data, partial=True)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)
```

Использование `get_or_create` гарантирует, что настройки существуют для каждого пользователя, даже если они не были созданы при регистрации (например, для пользователей, созданных через админку).

Все модели зарегистрированы в Django Admin с настройками отображения списка, фильтрации и поиска, что обеспечивает удобное администрирование данных.

5 РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ

5.1 Технологический стек frontend

Клиентская часть приложения реализована на языке TypeScript с использованием библиотеки React 18. Выбор TypeScript обусловлен тем, что статическая типизация позволяет выявлять ошибки на этапе компиляции, улучшает читаемость и поддерживаемость кода. TypeScript используется в строгом режиме (`strict: true`) с проверкой на неиспользуемые переменные и параметры.

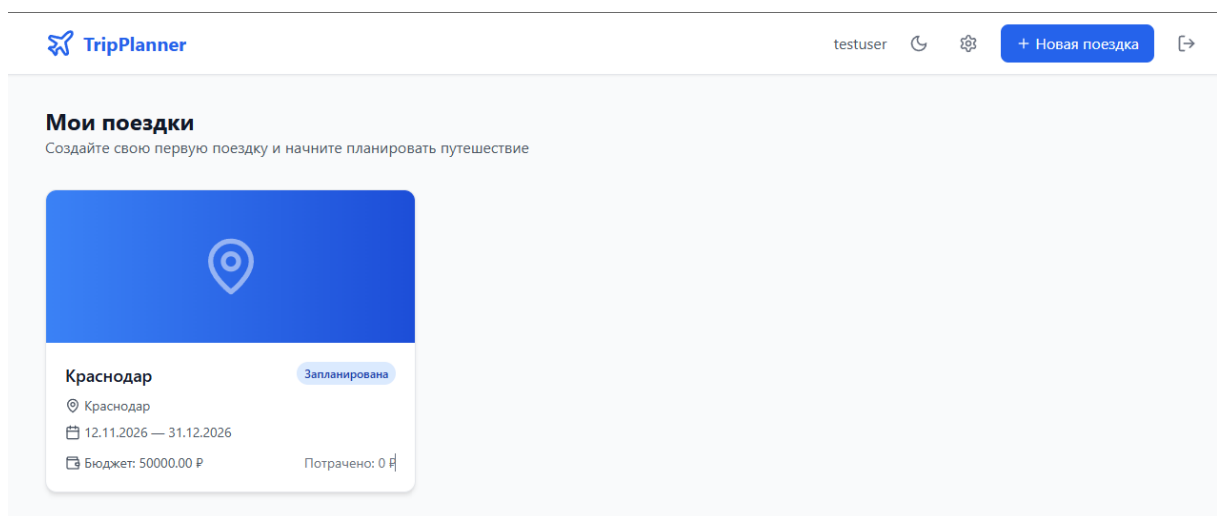


Рисунок 4 — Главная страница приложения

На рисунке 4 показана главная страница приложения в светлой теме оформления. На ней отображается список запланированных поездок пользователя с краткой информацией о каждой поездке.

React — библиотека для построения пользовательских интерфейсов, основанная на компонентном подходе. Компоненты инкапсулируют разметку, стили и логику, что упрощает повторное использование кода. React использует виртуальный DOM, что обеспечивает высокую эффективность обновлений.

Для сборки проекта используется Vite 5 — современный сборщик, обеспечивающий высокую скорость разработки за счёт использования нативных ES-модулей. В режиме разработки Vite обеспечивает мгновенное

обновление модулей (HMR). Конфигурация Vite настраивает dev-сервер на порту 3000 и проксирование запросов к /api/ на backend (localhost:8000), что обходить ограничения CORS в режиме разработки.

Для маршрутизации используется react-router-dom версии 6. Эта библиотека обеспечивает навигацию между экранами без перезагрузки страницы. Поддерживаются вложенные маршруты, параметры URL и защита маршрутов через PrivateRoute.

Для выполнения HTTP-запросов используется axios — популярный HTTP-клиент. Axios предоставляет удобный API для настройки интерцепторов, которые используются для автоматического добавления JWT-токена к запросам.

Для стилизации применяется Tailwind CSS 3 — utility-first CSS-фреймворк. Tailwind предоставляет набор готовых CSS-классов, которые комбинируются непосредственно в разметке компонентов. Конфигурация Tailwind определяет кастомную палитру primary (синие оттенки) и включает darkMode: "class" для поддержки тёмной темы.

Для иконок используется lucide-react — библиотека SVG-иконок.

Структура frontend-проекта:

```
frontend/src/
├── App.tsx           # маршрутизация и провайдеры
├── main.tsx          # точка входа
├── index.css          # Tailwind-директивы
├── components/
│   ├── Layout.tsx    # каркас страницы
│   ├── PrivateRoute.tsx # защита маршрутов
│   └── TripCard.tsx   # карточка поездки
├── context/
│   ├── AuthContext.tsx # авторизация (JWT)
│   ├── ThemeContext.tsx # тема (light/dark)
│   └── I18nContext.tsx  # локализация (ru/en)
├── hooks/
│   └── useSettings.ts  # загрузка настроек
├── pages/
│   ├── Login.tsx
│   ├── Register.tsx
│   ├── Trips.tsx       # список поездок
│   ├── TripDetail.tsx  # детальная страница
│   ├── TripForm.tsx    # создание/редактирование
│   └── Settings.tsx    # настройки
```

```
└─ services/api.ts      # axios + JWT-интерцепторы
└─ utils/format.ts     # форматирование валюты
```

5.2 Реализация детальной страницы поездки

Детальная страница поездки (TripDetail.tsx) является центральным элементом пользовательского интерфейса. Она содержит шесть вкладок, переключаемых пользователем: маршрут (itinerary), расходы (expenses), чемодан (packing), бронирования (bookings), карта (map) и рекомендации (recommendations).

A screenshot of a mobile application interface for creating a new trip. At the top left, there is a back arrow and the text "Отмена" (Cancel). The main title "Новая поездка" (New Trip) is centered at the top of the form. Below the title are several input fields: "Название" (Name) with a single-line text input; "Направление" (Direction) with a single-line text input; "Описание" (Description) with a multi-line text area; "Начало" (Start) and "Окончание" (End) with date pickers showing "ДД.ММ.ГГГГ"; "Бюджет" (Budget) with a single-line text input; and "Статус" (Status) with a dropdown menu currently showing "Запланирована" (Planned). At the bottom left of the form is a blue button with a floppy disk icon and the text "Сохранить" (Save).

Рисунок 5 — Форма добавления и редактирования поездки

На рисунке 5 изображена форма создания новой поездки, в которой пользователь указывает название, направление, описание, даты начала и окончания, бюджет и статус поездки. Аналогичная форма используется и для редактирования существующей поездки.

При загрузке страницы выполняется запрос к API для получения детальной информации о поездке, включая все дочерние элементы (itinerary_items, expenses, packing_items, bookings). Полученные данные сохраняются в локальном состоянии компонента:

```
useEffect(() => {
  if (id) {
    api.get(`/trips/${id}/`).then((res) => {
      setTrip(res.data)
      setItinerary(res.data.itinerary_items || [])
      setExpenses(res.data.expenses || [])
      setPacking(res.data.packing_items || [])
      setBookings(res.data.bookings || [])
    })
  }
}, [id])
```

Шапка страницы отображает название поездки, направление (с иконкой MapPin), даты (с иконкой Calendar), статус-бейдж с цветовым кодированием (planned — синий, active — зелёный, completed — серый, cancelled — красный), кнопки редактирования и удаления, а также блок бюджета с суммой расходов и запланированным бюджетом, отформатированными в выбранной валюте.

Вкладка «Маршрут» предоставляет форму добавления пунктов (дата, время, название, место) и список существующих пунктов, отсортированных по дате и времени. Добавление выполняется через POST-запрос к /api/itinerary/.

Вкладка «Расходы» предоставляет форму добавления расходов (название, сумма, категория из шести вариантов, дата) и список расходов с общей суммой. Категории имеют цветовое кодирование. При добавлении расхода обновляется вычисляемое поле total_expenses, что позволяет пользователю видеть актуальное отношение потраченной суммы к бюджету.

Вкладка «Чемодан» предоставляет поле ввода названия вещи и список с чекбоксами для отметки собранных. При отметке чекбокса выполняется PATCHN-запрос к `/api/packing/{id}/` для обновления поля `is_packed`. Собранные вещи отображаются с зачёркиванием и зелёной галочкой.

Вкладка «Бронирования» — наиболее сложная вкладка. Содержит блок «Переслать письмо» с `textarea` и кнопкой «Спарсить письмо» (`POST /api/parse-booking-email/`), форму добавления бронирования (тип: `flight/hotel/train/car/other`, название, номер подтверждения, место, цена, даты) и список бронирований с иконками по типу (`Plane, Hotel, Train, Car, FileText` из `lucide-react`).

Вкладка «Карта» отображает плейсхолдер с приглашением добавить Google Maps API-ключ в настройках. В модели `UserSettings` предусмотрено поле `google_maps_api_key`, а на странице настроек — поле ввода для ключа. Полная интеграция картографического сервиса требует подключения внешнего API.

Вкладка «Рекомендации» предоставляет поле ввода предпочтений (например, «исторические места, вегетарианские рестораны») и кнопку «Получить рекомендации». Текущая реализация генерирует шаблонные рекомендации на основе направления поездки. Полная интеграция с AI требует подключения внешнего API (`OpenAI, Google Places`).

5.3 Темы оформления и локализация

Приложение поддерживает две темы оформления: светлую и тёмную. Реализация тем основана на классе `dark` на корневом элементе документа и CSS-классах `Tailwind` с префиксом `dark:`.

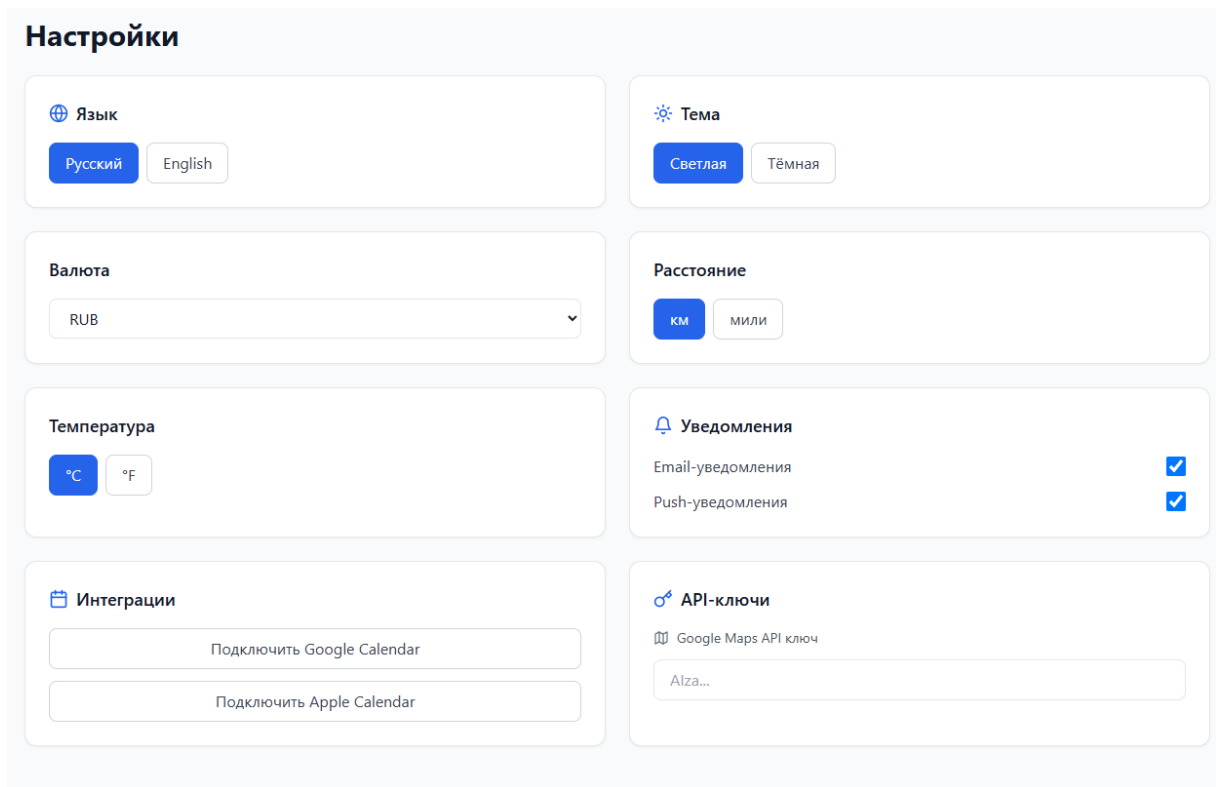


Рисунок 6 — Страница настроек с выбором темы оформления

На рисунке 6 представлена страница настроек пользователя. Здесь можно переключать язык интерфейса, выбирать светлую или тёмную тему оформления, изменять валюту, единицы измерения и настроить уведомления.

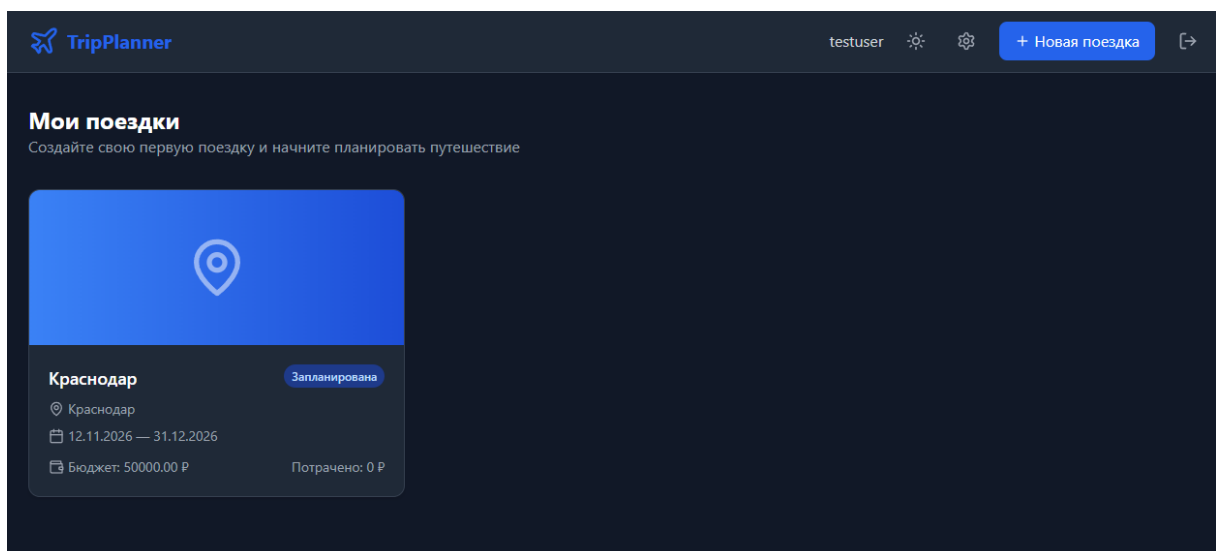


Рисунок 7 — Главная страница в тёмной теме оформления

На рисунке 7 показан вид главной страницы приложения в тёмной теме оформления. Переключение темы выполняется через страницу настроек и применяется ко всему интерфейсу.

Управление темой реализовано через React Context (ThemeContext). Тема хранится в localStorage и синхронизируется с настройками на сервере. При изменении темы класс dark добавляется или удаляется на элементе `<html>`, что активирует соответствующие CSS-правила. Переключатель темы расположен в навигационной панели (иконки Moon и Sun из lucide-react).

```
const [theme, setThemeState] = useState<Theme>(() => {
  const saved = localStorage.getItem("theme") as Theme
  return saved || "light"
})
useEffect(() => {
  const root = window.document.documentElement
  if (theme === "dark") root.classList.add("dark")
  else root.classList.remove("dark")
  localStorage.setItem("theme", theme)
}, [theme])
```

Локализация приложения реализована через I18nContext — React Context, предоставляющий функцию перевода `t(key)` и текущий язык. Поддерживаются два языка: русский (ru) и английский (en). Словари содержат около 100 ключей каждый, включая вложенные объекты для статусов поездок, категорий расходов и типов бронирований.

```
const t = (key: string, nestedKey?: string): string => {
  const value = translations[language][key]
  if (nestedKey && typeof value === "object") {
    return value[nestedKey] || nestedKey
  }
  return (value as string) || key
}
```

Выбранный язык сохраняется в localStorage и синхронизируется с настройками на сервере (через PATCH /api/auth/settings/). Значение по умолчанию — русский.

Дополнительно реализованы утилиты форматирования валюты, расстояния и температуры (utils/format.ts). Функция formatCurrency форматирует сумму в выбранной валюте с использованием

Intl.NumberFormat. Это обеспечивает корректное отображение символа валюты и разделителей разрядов в зависимости от выбранной валюты.

5.4 Контейнеризация и развёртывание

Для обеспечения воспроизводимости сборки и удобства развёртывания приложение упаковано в Docker-контейнеры. Используется многоэтапная сборка для frontend-контейнера.

Dockerfile для frontend содержит два этапа. На первом этапе (FROM node:20-alpine) выполняется установка зависимостей и сборка production-версии (npm run build). На втором этапе (FROM nginx:alpine) скопированные статические файлы помещаются в директорию, обслуживаемую Nginx.

Dockerfile для backend основан на образе python:3.10-slim. В образ устанавливаются зависимости из requirements.txt, копируется исходный код. Запуск выполняет автоматическое применение миграций и запуск gunicorn.

Для оркестрации используется Docker Compose с тремя конфигурациями:

- docker-compose.yml — для разработки (runserver, DEBUG=True, монтирование исходного кода);
- docker-compose.prod.yml — для production (gunicorn, DEBUG=False, SSL-сертификаты);
- docker-compose.shared-proxy.yml — для работы за внешним reverse-proxy.

Nginx в production выполняет функции обратного прокси: терминирует SSL-соединения с сертификатами Let's Encrypt, раздаёт статические файлы frontend, проксирует запросы к /api/ на backend-контейнер и раздаёт загруженные медиафайлы. Директива try_files \$uri \$uri /index.html обеспечивает SPA-fallback для react-router.

Развёртывание на VPS-сервере: установка Docker и Nginx, загрузка исходного кода, настройка DNS, получение SSL-сертификатов Let's

Encrypt, настройка Nginx, запуск контейнеров. Приложение доступно по адресу <https://travelplanning2026.ru>.

6 ТЕСТИРОВАНИЕ

6.1 Методика тестирования

Тестирование является важным этапом разработки программного обеспечения, направленным на проверку соответствия разработанного приложения требованиям технического задания. В рамках курсовой работы проводилось функциональное тестирование, представляющее собой проверку работы отдельных функций и сценариев использования.

Функциональное тестирование выполнялось вручную путём выполнения типичных сценариев работы пользователя в развёрнутом приложении. Для каждого сценария проверялось соответствие фактического результата ожидаемому. Тестирование проводилось в production-окружении, развёрнутом на VPS-сервере.

В процессе тестирования проверялись следующие группы сценариев: регистрация и аутентификация, управление поездками, работа с вкладками детальной страницы, настройки пользователя, темы оформления и локализация, развёртывание в Docker.

6.2 Результаты функционального тестирования

Сценарий 1. Регистрация пользователя. На странице регистрации вводились логин, email и пароль. Ожидаемый результат: успешная регистрация, автоматический вход, создание настроек по умолчанию. Фактический результат: соответствует. Сценарий пройден.

Таблица 3 — Результаты функционального тестирования

№	Тест-кейс	Ожидаемый результат	Фактический результат	Статус
---	-----------	---------------------	-----------------------	--------

1	Регистрация пользователя	Пользователь создан	Успешно	Пройден
2	Авторизация по JWT	Выдан токен	Успешно	Пройден
3	Создание поездки	Поездка добавлена	Успешно	Пройден
4	Добавление маршрута	Точка маршрута сохранена	Успешно	Пройден
5	Добавление расхода	Расход учтён	Успешно	Пройден
6	Список вещей	Предмет добавлен	Успешно	Пройден

Сценарий 2. Вход пользователя. На странице входа вводились логин и пароль. Ожидаемый результат: успешный вход, получение JWT-токенов, перенаправление на список поездок. Фактический результат: соответствует. Сценарий пройден.

Сценарий 3. Вход с неверными данными. Вводились некорректные логин и/или пароль. Ожидаемый результат: сообщение об ошибке. Фактический результат: соответствует. Сценарий пройден.

Сценарий 4. Создание поездки. На форме создания заполнялись название, направление, даты, бюджет. Ожидаемый результат: поездка создаётся, отображается в списке. Фактический результат: соответствует. Сценарий пройден.

Сценарий 5. Редактирование поездки. На форме редактирования изменялись название и бюджет. Ожидаемый результат: изменения сохраняются. Фактический результат: соответствует. Сценарий пройден.

Сценарий 6. Удаление поездки. Нажималась кнопка удаления поездки. Ожидаемый результат: поездка удаляется вместе со всеми связанными данными (маршрут, расходы, чемодан, бронирования). Фактический результат: соответствует. Сценарий пройден.

Сценарий 7. Добавление пункта маршрута. На вкладке «Маршрут» заполнялись дата, время, название, место. Ожидаемый результат: пункт добавляется в список. Фактический результат: соответствует. Сценарий пройден.

Сценарий 8. Добавление расхода. На вкладке «Расходы» заполнялись название, сумма, категория, дата. Ожидаемый результат: расход добавляется, общая сумма обновляется. Фактический результат: соответствует. Сценарий пройден.

Сценарий 9. Проверка бюджета. Добавлялись расходы, превышающие бюджет. Ожидаемый результат: отображение суммы расходов больше бюджета. Фактический результат: соответствует. Сценарий пройден.

Сценарий 10. Добавление вещи в чемодан. На вкладке «Чемодан» вводилось название вещи. Ожидаемый результат: вещь добавляется в список. Фактический результат: соответствует. Сценарий пройден.

Сценарий 11. Отметка вещи как упакованной. Нажимался чекбокс рядом с вещью. Ожидаемый результат: чекбокс отмечен, вещь отображается как собранная. Фактический результат: соответствует. Сценарий пройден.

Сценарий 12. Добавление бронирования. На вкладке «Бронирования» заполнялись тип, название, номер подтверждения, цена. Ожидаемый результат: бронирование добавляется с соответствующей иконкой. Фактический результат: соответствует. Сценарий пройден.

Сценарий 13. Парсинг email. На вкладке «Бронирования» вводился текст письма и нажималась кнопка «Спарсить письмо». Ожидаемый результат: создаётся бронирование-заглушка. Фактический результат: соответствует. Сценарий пройден.

Сценарий 14. Изоляция данных. Создавалась поездка под пользователем А, затем выполнялся вход под пользователем В. Ожидаемый результат: пользователь В не видит поездку пользователя А. Фактический результат: соответствует. Сценарий пройден.

Сценарий 15. Переключение темы. В навигационной панели нажималась кнопка переключения темы. Ожидаемый результат: тема мгновенно меняется между светлой и тёмной, сохраняется после перезагрузки. Фактический результат: соответствует. Сценарий пройден.

Сценарий 16. Переключение языка. На странице настроек выбирался английский язык. Ожидаемый результат: интерфейс переводится на английский, выбор сохраняется. Фактический результат: соответствует. Сценарий пройден.

Сценарий 17. Смена валюты. На странице настроек выбиралась валюта USD. Ожидаемый результат: суммы отображаются в долларах. Фактический результат: соответствует. Сценарий пройден.

Сценарий 18. Защита приватных маршрутов. Выполнялся выход из системы, затем вводился URL главной страницы. Ожидаемый результат: перенаправление на страницу входа. Фактический результат: соответствует. Сценарий пройден.

Сценарий 19. Развёртывание в Docker. Приложение разворачивалось в Docker-контейнерах на VPS. Ожидаемый результат: приложение доступно по адресу <https://travelplanning2026.ru>. Фактический результат: соответствует. Сценарий пройден.

Сценарий 20. Адаптивность. Страницы открывались на экранах различной ширины. Ожидаемый результат: корректное отображение на всех устройствах. Фактический результат: соответствует. Сценарий пройден.

Таким образом, в результате функционального тестирования все 20 проверенных сценариев работают корректно. Разработанное приложение полностью соответствует сформулированным требованиям.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано полноценное веб-приложение для планирования туристических поездок, включающее клиентскую и серверную части, систему аутентификации, инструменты для составления маршрута, учёта расходов и формирования списка вещей, а также поддержку бронирований, карты и рекомендаций. Все поставленные задачи были выполнены в полном объёме.

В ходе работы были достигнуты следующие результаты:

- проведён анализ предметной области и обзор существующих решений;
- сформулированы функциональные и нефункциональные требования;
- спроектирована клиент-серверная архитектура с разделением backend и frontend;
- спроектированы шесть моделей данных и REST API из 11 endpoints;
- реализован backend на Python 3.10 с использованием Django 5 и DRF;
- реализован frontend на TypeScript с использованием React 18 и Vite;
- реализована детальная страница поездки с шестью вкладками;
- организована JWT-аутентификация с access и refresh токенами;
- реализованы тёмная и светлая темы оформления;
- реализована локализация (русский и английский языки);
- реализован выбор валюты, единиц расстояния и температуры;
- выполнена контейнеризация с Docker и Docker Compose;
- выполнено развёртывание на VPS с Nginx и SSL-сертификатами;
- проведено функциональное тестирование (20 сценариев), подтвердившее корректную работу.

Применённый технологический стек (Python, Django, DRF, TypeScript, React, Vite, Tailwind CSS, Docker, Nginx) является современным и широко распространённым. Разделение приложения на независимые backend и frontend упростило разработку и тестирование.

Разработанное приложение может быть использовано в качестве готового решения для планирования личных и командных поездок, а также в качестве учебного примера построения современных клиент-серверных веб-приложений. Приложение развёрнуто в production-окружении и доступно по адресу <https://travelplanning2026.ru>.

Перспективы дальнейшего развития проекта могут включать: интеграцию картографического сервиса (Google Maps, Яндекс.Карты), реализацию real-time обновлений с использованием WebSocket (Django Channels), подключение AI для генерации рекомендаций, реализацию парсинга email-подтверждений бронирований, добавление совместной работы над поездками, переход на PostgreSQL, разработку мобильного приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Django Documentation. — URL: <https://docs.djangoproject.com/> (дата обращения: 16.06.2026).
2. Django REST Framework Documentation. — URL: <https://www.django-rest-framework.org/> (дата обращения: 16.06.2026).
3. React Documentation. — URL: <https://react.dev/> (дата обращения: 16.06.2026).
4. TypeScript Documentation. — URL: <https://www.typescriptlang.org/> (дата обращения: 16.06.2026).
5. Tailwind CSS Documentation. — URL: <https://tailwindcss.com/> (дата обращения: 16.06.2026).
6. Docker Documentation. — URL: <https://docs.docker.com/> (дата обращения: 16.06.2026).
7. Vite Documentation. — URL: <https://vitejs.dev/> (дата обращения: 16.06.2026).
8. Официальная документация Nginx. — URL: <https://nginx.org/en/docs/> (дата обращения: 16.06.2026).
9. Certbot / Let's Encrypt. — URL: <https://certbot.eff.org/> (дата обращения: 16.06.2026).
10. JSON Web Token (JWT). RFC 7519. — URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 16.06.2026).
11. Документация django-rest-framework-simplejwt. — URL: <https://github.com/jazzband/django-rest-framework-simplejwt> (дата обращения: 16.06.2026).
12. Документация react-router-dom. — URL: <https://reactrouter.com/> (дата обращения: 16.06.2026).
13. Документация axios. — URL: <https://axios-http.com/> (дата обращения: 16.06.2026).

14. Документация lucide-react. — URL: <https://lucide.dev/> (дата обращения: 16.06.2026).
15. Fielding R. Architectural Styles and the Design of Network-based Software Architectures. — 2000.
16. Документация Pillow. — URL: <https://pillow.readthedocs.io/> (дата обращения: 16.06.2026).